
ChatGPT Interaction

<https://chatgpt.com/share/6ac90171-499c-83e8-96c0-15a39a124be2>

I need to implement the features listed in the README of my 51_rhythm-tap project. Please implement them one at a time and keep each task separate.

Worked for 14s

I need to implement the features listed in the README of my 51_rhythm-tap project. Please implement them one at a time and keep each task separate.

We'll start from the beginning, with the original project and no changes implemented yet. For each task, we'll finish that task before moving on to the next one, and I'll include the relevant code immediately after each implementation.

Please share the repository link or upload the project ZIP containing the README and source files so we can start with **Task 1**.

README (2).mdDocument

https://github.com/SETAPESU26/51_rhythm-tap

Add long notes that the player must hold for 1 second.

Requirements:

- Add hold notes.
- Make them visually longer than normal notes.
- The corresponding key must remain pressed for 1 second.
- Preserve normal tap notes.
- Preserve Task 1 sound effects.
- Do not add external dependencies.

Worked for 21s

Task 2: Hold Notes

We'll implement this as a separate change on top of Task 1, preserving the existing sound effects and normal tap notes.

Implementation approach

- Normal notes remain unchanged.
- Hold notes have a longer visual body.
- Press the corresponding lane key when the hold note reaches the hit line.
- Keep the key pressed for **1,000 milliseconds** to earn the points.

- Releasing early counts as a miss.
- Successful hold notes play the existing PERFECT sound.
- No new dependencies are required.

We'll modify two files: `game/beat.py` and `game/game_engine.py`.

1. Update `game/beat.py`

Replace the file with this version. It adds hold-note properties while keeping the existing constructor behavior compatible with normal notes.

```
import pygame
import random

LANES = 4

LANE_KEYS = [pygame.K_d, pygame.K_f, pygame.K_j, pygame.K_k]
LANE_LABELS = ["D", "F", "J", "K"]
LANE_COLORS = [
    (220, 80, 80),
    (80, 180, 220),
    (100, 220, 100),
    (220, 180, 60),
]

class Note:
    WIDTH = 70
    HEIGHT = 20
    HOLD_HEIGHT = 80
    HOLD_DURATION_MS = 1000

    def __init__(self, lane, y=-30, speed=4, is_hold=False):
        self.lane = lane
        self.y = y
```

```
self.speed = speed  
self.is_hold = is_hold
```

```
self.hit = False  
self.missed = False
```

```
# Hold-note state.  
self.holding = False  
self.hold_started_at = None
```

```
def update(self):  
    self.y += self.speed
```

2. Update game/game_engine.py

Make the following changes to the Task 1 version of your file.

A. Update spawn_note()

Replace the existing method with:

```
def spawn_note(self):  
    lane = random.randint(0, LANES - 1)  
  
    # Task 2: 25% chance of spawning a hold note.  
    is_hold = random.random() < 0.25  
  
    self.notes.append(  
        Note(  
            lane,  
            y=-30,  
            speed=self.speed,  
            is_hold=is_hold,  
        )  
    )
```

B. Update handle_events()

Add key-release detection so the game can detect when a hold is released. Replace the method with:

```
def handle_events(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            return False

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_r:
                self.reset()
            elif not self.game_over:
                for i, key in enumerate(LANE_KEYS):
                    if event.key == key:
                        self.process_tap(i)

        elif event.type == pygame.KEYUP:
            if not self.game_over:
                for i, key in enumerate(LANE_KEYS):
                    if event.key == key:
                        self.release_hold(i)

    return True
```

C. Update process_tap()

Keep the existing grade and scoring logic for normal notes. In the note-selection loop, replace the condition with:

```
if (
    note.lane == lane
    and not note.hit
```

```
and not note.missed
```

```
and not note.holding
```

```
):
```

Then, inside the successful-hit branch, before the existing `best.hit = True` statement, insert:

Task 2: Start a hold instead of judging it as a tap.

```
if best.is_hold:
```

```
    best.holding = True
```

```
    best.hold_started_at = pygame.time.get_ticks()
```

```
    self.feedback.append(
```

```
        ["HOLD", (100, 220, 255), 40, lane_x, HIT_Y - 30]
```

```
)
```

```
    return
```

Normal notes will continue through the existing PERFECT, GREAT, and OK scoring and sound-effect logic.

D. Add the hold-completion methods

Insert these methods into `GameEngine`, at the same indentation level as `process_tap()`:

```
def finish_hold(self, note, success):
```

```
    """Complete a hold note or mark an early release as a miss."""
```

```
    if not note.holding:
```

```
        return
```

```
    note.holding = False
```

```
    lane_x = note.lane * LANE_W + LANE_W // 2
```

```
    if success:
```

```
        note.hit = True
```

```
        self.combo += 1
```

```
        self.max_combo = max(self.max_combo, self.combo)
```

```
self.score += 300 * max(1, self.combo // 5)
```

```
# Preserve Task 1 sound effects.
```

```
if self.audio_enabled:
```

```
    sound = self.hit_sounds.get("PERFECT")
```

```
    if sound:
```

```
        sound.play()
```

```
self.feedback.append(
```

```
    ["PERFECT", (255, 220, 0), 40, lane_x, HIT_Y - 30]
```

```
)
```

```
else:
```

```
    note.missed = True
```

```
    self.combo = 0
```

```
self.feedback.append(
```

```
    ["MISS", (220, 60, 60), 40, lane_x, HIT_Y - 30]
```

```
)
```

```
def release_hold(self, lane):
```

```
    """Judge a hold when the corresponding key is released."""
```

```
E. Update update()
```

Inside the existing loop that updates each note, immediately after `note.update()`, add:

```
# Task 2: Award the hold once one full second has elapsed.
```

```
if note.is_hold and note.holding:
```

```
    elapsed = (
```

```
        pygame.time.get_ticks() - note.hold_started_at
```

```
)
```

```
if elapsed >= Note.HOLD_DURATION_MS:
    self.finish_hold(note, success=True)
```

The KEYUP handler handles early releases. The timer uses elapsed milliseconds rather than counting frames, so the requirement is one actual second rather than an assumed 60 frames.

F. Update the note drawing section in draw()

Keep the existing note drawing code, but replace the notes loop with:

Task 2: Draw normal notes and longer hold notes.

```
for note in self.notes:
```

```
    if note.hit:
        continue
```

```
    lx = note.lane * LANE_W + LANE_W // 2
    rect = note.get_rect(lx)
```

```
    pygame.draw.rect(
        self.screen,
        LANE_COLORS[note.lane],
        rect,
        border_radius=5,
    )
```

Show the hold timer filling the note while the key is held.

```
if note.is_hold and note.holding:
```

```
    elapsed = (
        pygame.time.get_ticks() - note.hold_started_at
    )
    progress = min(
        1.0,
        elapsed / Note.HOLD_DURATION_MS,
```

```
)  
progress_height = int(rect.height * progress)
```

```
pygame.draw.rect(  
    self.screen,  
    (255, 255, 255),  
    pygame.Rect(  
        rect.x,  
        rect.bottom - progress_height,  
        rect.width,  
        progress_height,  
    ),
```

3. Test Task 2

Run the game:

```
python main.py
```

Verify the following checklist:

Normal notes still score using PERFECT, GREAT, and OK.

Hold notes appear visibly longer than normal notes.

Holding the correct key for one second scores a PERFECT and plays the existing sound.

Releasing before one second marks the hold as a MISS.

The game still runs without additional dependencies.

Restarting with R still works.

0 of 6 checks completed

Once the tests pass, commit this task separately:

```
git add game/beat.py game/game_engine.py
```

```
git commit -m "Add one-second hold notes"
```

Status: Task 2 code changes are specified above, but they have not been executed or tested against your local repository. We should verify these changes before moving on to Task 3.

before this

Add a short beep/drum sound whenever the player successfully hits a note with a PERFECT, GREAT, or OK rating.

Requirements:

- Sound on PERFECT.
- Sound on GREAT.
- Sound on OK.
- No sound for MISS.
- Preserve the existing gameplay and scoring.
- Do not add external dependencies.

Task 1: Sound Effects on Hit

We'll implement this task first, before Hold Notes, starting from the original 51_rhythm-tap code.

Requirements

- Play a sound on PERFECT.
- Play a sound on GREAT.
- Play a sound on OK.
- No sound on MISS.
- Preserve existing gameplay and scoring.
- Add no external dependencies.

Code changes — game/game_engine.py

1. Add imports

At the top of the file, add math and array:

```
import math
```

```
from array import array
```

2. Initialize audio safely

In `GameEngine.__init__()`, replace the existing `pygame.mixer.init()` call with:

```
# Task 1: Sound effects on successful hits.
```

```
self.audio_enabled = False
```

```
self.hit_sounds = {}
```

```
try:
```

```
    pygame.mixer.init(
        frequency=44100,
        size=-16,
        channels=1,
```

```
        buffer=512
    )
```

```
self.audio_enabled = True
```

```
self.hit_sounds = {
    "PERFECT": self._create_hit_sound(880, 0.08),
    "GREAT": self._create_hit_sound(660, 0.08),
    "OK": self._create_hit_sound(440, 0.08),
}
```

```
except pygame.error:
```

```
    # Continue playing even if audio is unavailable.
```

```
    self.audio_enabled = False
```

Keep the rest of the constructor unchanged.

3. Add the sound-generation method

Add this method inside GameEngine:

```
def _create_hit_sound(self, frequency, duration):
    """Generate a short beep without external audio files."""
    sample_rate = 44100
    sample_count = int(sample_rate * duration)
    samples = array("h")

    for i in range(sample_count):
        t = i / sample_rate
        envelope = 1.0 - (i / sample_count)

        sample = int(
            12000
            * envelope
            * math.sin(2 * math.pi * frequency * t)
```

)

```
samples.append(sample)
```

```
return pygame.mixer.Sound(buffer=samples.tobytes())
```

This generates short tones in memory using Python's built-in math and array modules and Pygame's existing audio support.

4. Play the sound after a successful hit

In `process_tap()`, find the section where the game determines grade and pts, then updates the score:

```
self.combo += 1
```

```
self.max_combo = max(self.max_combo, self.combo)
```

```
self.score += pts * max(1, self.combo // 5)
```

Immediately after it, add:

```
# Task 1: Play sound only for successful hits.
```

```
if self.audio_enabled:
```

```
    sound = self.hit_sounds.get(grade)
```

```
    if sound is not None:
```

```
        sound.play()
```

Do not add this code to the MISS branch. Consequently, misses remain silent, and the existing score and combo calculations are unchanged.

Test Task 1

Run the game from your project directory:

```
python main.py
```

Check that:

- PERFECT, GREAT, and OK hits play their respective tones.
- MISS produces no sound.
- Scoring, combo, note movement, restart, and game-over behavior remain unchanged.
- The game does not crash if audio initialization fails.

Once verified, commit Task 1 separately:

```
git add game/game_engine.py
```

```
git commit -m "Add sound effects for successful hits"
```

Status: The implementation is provided, but it has not been run in your local environment. After you verify Task 1, we'll proceed to Task 2: Hold Notes, preserving these sound effects.

now do task 2

Worked for 9s

Task 2: Hold Notes

We'll now add one-second hold notes while preserving **Task 1's PERFECT, GREAT, and OK sound effects**, normal tap notes, existing scoring, and gameplay controls.

No external dependencies are required.

Code changes

1. Update game/beat.py

Replace the existing Note class with the following. Keep any other constants or classes in the file that your project uses.

class Note:

 WIDTH = 70

 HEIGHT = 20

 HOLD_HEIGHT = 100

 HOLD_DURATION_MS = 1000

 def __init__(self, lane, y=-30, speed=4, is_hold=False):

 self.lane = lane

 self.y = y

 self.speed = speed

 self.is_hold = is_hold

 self.hit = False

 self.missed = False

 # Hold-note state

 self.holding = False

 self.hold_started_at = None

 def update(self):

 # Keep a note stationary while the player holds it.

```

if not self.holding:
    self.y += self.speed

def get_rect(self, lane_x):
    height = self.HOLD_HEIGHT if self.is_hold else self.HEIGHT
    return pygame.Rect(
        lane_x - self.WIDTH // 2,
        int(self.y),
        self.WIDTH,
        height
    )

```

The `is_hold` flag distinguishes long notes from normal tap notes. Hold notes are drawn 100 pixels tall, while normal notes retain their original height.

2. Update game/game_engine.py

A. Spawn both types of notes

Replace the existing `spawn_note()` method with:

```

def spawn_note(self):
    lane = random.randint(0, LANES - 1)

    # Spawn hold notes occasionally; otherwise spawn normal notes.
    is_hold = random.random() < 0.25

    self.notes.append(
        Note(
            lane,
            y=-30,
            speed=self.speed,
            is_hold=is_hold
        )
    )

```

This makes approximately 25% of newly spawned notes hold notes. The remaining 75% are normal notes.

Make sure random is imported and Note and LANES are already available in this file.

B. Detect when a hold key is released

Update `handle_events()` so that it handles both `KEYDOWN` and `KEYUP`. Preserve the existing quit and restart handling.

```
def handle_events(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            self.running = False

        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_r:
                self.reset()

            elif event.key in LANE_KEYS:
                lane = LANE_KEYS.index(event.key)
                self.process_tap(lane)

        elif event.type == pygame.KEYUP:
            if event.key in LANE_KEYS:
                lane = LANE_KEYS.index(event.key)
                self.release_hold(lane)
```

If your current event handler has additional logic, retain it rather than deleting it.

C. Start holding a note in `process_tap()`

In your existing `process_tap()` method, keep the current note-selection and hit-window logic. When the best matching note has been identified and is within the hit window, insert this check **before marking the note as hit or awarding normal tap points**:

```
if best.is_hold:
    best.holding = True
    best.hold_started_at = pygame.time.get_ticks()

self.feedback.append([
    "HOLD",
```

```

        (100, 220, 255),

        40,

        lane_x,

        HIT_Y - 30

    ])

    return

```

Also ensure your candidate-note selection excludes notes that are already being held:

and not note.holding

Keep the existing scoring, grade calculation, and Task 1 sound playback for normal tap notes unchanged.

D. Add hold completion and early-release handling

Add these two methods inside GameEngine:

```

def finish_hold(self, note, success):

    if not note.holding:

        return

    note.holding = False

    lane_x = note.lane * LANE_W + LANE_W // 2

    if success:

        note.hit = True

        self.combo += 1

        self.max_combo = max(self.max_combo, self.combo)

        self.score += 300 * max(1, self.combo // 5)

    # Reuse Task 1's existing PERFECT sound.

    if self.audio_enabled:

        sound = self.hit_sounds.get("PERFECT")

        if sound is not None:

            sound.play()

```

```

self.feedback.append([
    "PERFECT",
    (255, 220, 0),
    40,
    lane_x,
    HIT_Y - 30
])
else:
    note.missed = True
    self.combo = 0

```

```

self.feedback.append([
    "MISS",
    (220, 60, 60),
    40,
    lane_x,
    HIT_Y - 30

```

A hold succeeds after one second. Releasing early marks it as a miss and resets the combo. Successful holds reuse Task 1's existing PERFECT sound.

E. Update the note loop

In `update()`, make sure a note does not move while being held, and complete the hold after one second.

Replace the relevant note-update portion with:

```
for note in self.notes:
```

```
    note.update()
```

```
    if note.is_hold and note.holding:
```

```
        elapsed = (
            pygame.time.get_ticks() - note.hold_started_at
        )
```

```
        if elapsed >= Note.HOLD_DURATION_MS:
```

```

        self.finish_hold(note, success=True)

    elif (
        not note.hit
        and not note.missed
        and not note.holding
        and note.y > HIT_Y + HIT_WINDOW + Note.HEIGHT
    ):
        note.missed = True
        self.misses += 1
        self.combo = 0

```

Retain the rest of your existing update logic, including feedback updates and removal of completed or missed notes.

Important: The `not note.holding` condition prevents the normal missed-note logic from incorrectly marking a note as missed during its one-second hold.

F. Render hold notes differently

In the drawing section, replace the existing note-drawing loop with this pattern, retaining your existing lane-position and color variables:

for note in self.notes:

```

    lx = note.lane * LANE_W + LANE_W // 2
    rect = note.get_rect(lx)

```

```

    color = LANE_COLORS[note.lane]

```

if note.is_hold:

```

    pygame.draw.rect(
        self.screen,
        color,
        rect,
        border_radius=8
    )

```

```

if note.holding:
    elapsed = (
        pygame.time.get_ticks() - note.hold_started_at
    )
    progress = min(
        1.0,
        elapsed / Note.HOLD_DURATION_MS
    )

    progress_height = int(rect.height * progress)

    pygame.draw.rect(
        self.screen,
        (255, 255, 255),
        pygame.Rect(
            rect.x,
            rect.bottom - progress_height,
            rect.width,
            progress_height
        ),
        border_radius=8
    )

```

If your screen surface is not named `self.screen`, use the existing surface variable. Likewise, retain your project's existing lane-color constants if their names differ.

How the feature works

Normal tap note

Press D, F, J, or K once in the hit window. Existing scoring and grade sounds remain intact.

Hold note

Press the corresponding key and keep it down for 1,000 milliseconds. The progress fill advances while the note stays in place.

Test Task 2

Run:

python main.py

Verify the following:

- Normal tap notes still work.
- Hold notes are visibly longer.
- Holding the correct key for one second awards a successful hold.
- Releasing before one second results in a miss.
- Successful holds play the existing PERFECT sound.
- Normal PERFECT, GREAT, and OK sounds still work; misses remain silent.
- Restart and existing gameplay still work.

Once tested, commit this task separately:

```
git add game/beat.py game/game_engine.py
```

```
git commit -m "Add one-second hold notes"
```

Status: Task 2 code is provided, but it has not been executed or verified against your local project. The snippets assume the method and constant names used in the earlier Task 1 implementation; check those names against your current files before replacing methods.

Task 3: BPM-Synced Spawning

Notes should spawn on exact beats at a given BPM.

Requirements:

- Default BPM: 120.
- At 120 BPM, one beat occurs every 0.5 seconds.
- Notes should spawn according to the BPM.
- Keep random lane selection.
- Preserve normal and hold notes.
- Preserve Tasks 1 and 2.
- Do not implement Task 4 yet.
- Do not add external dependencies.

Worked for 13s

Implemented beat timing

Reviewed rhythm-tap code and isolated Task 3 spawning while preserving prior features

Task 3: BPM-Synced Spawning

We will modify note spawning so that notes appear according to a BPM-based schedule instead of a fixed frame counter. This preserves the existing random lane selection, normal notes, one-second hold notes, and Task 1 sound effects.

Task 3 scope: BPM-synced spawning only. Task 4 will not be implemented.

1. Add BPM configuration in game/game_engine.py

Inside `GameEngine.__init__`, add the following configuration. Keep your existing initialization code.

```
# Task 3: BPM-synced note spawning.self.bpm = 120self.beat_interval_ms = 60000.0 / self.bpm
```

At 120 BPM, this calculates an interval of 500 milliseconds per beat. You can change `self.bpm` to another positive BPM value when needed.

2. Schedule the first beat in reset()

Inside your existing `reset()` method, replace the old spawn-timer initialization, such as `self.spawn_timer = 0`, with:

```
# Task 3: Schedule note spawning against the beat clock.self.beat_interval_ms = 60000.0 / self.bpmself.next_beat_time = ( pygame.time.get_ticks() + self.beat_interval_ms)
```

This schedules the first note for half a second after the game resets at the default BPM.

3. Keep the existing spawn_note() method

Do not remove the Task 2 hold-note logic. The spawning method should continue to select a random lane and randomly choose between normal and hold notes.

It should retain this structure:

```
def spawn_note(self): lane = random.randint(0, LANES - 1) is_hold = random.random() < 0.25
self.notes.append( Note( lane, y=-30, speed=self.speed, is_hold=is_hold ) )
```

If your current method has additional necessary arguments or state, preserve them. The BPM change determines when this method is called, not how the notes are created.

4. Replace frame-based spawning in update()

Find the old logic that increments `self.spawn_timer` and compares it with `self.spawn_interval`.

Remove that spawning logic and replace it with:

```
# Task 3: Spawn notes on the BPM beat schedule.current_time = pygame.time.get_ticks()while
current_time >= self.next_beat_time: self.spawn_note() self.next_beat_time +=
self.beat_interval_ms
```

Keep the rest of `update()` unchanged, especially the existing hold-note completion, early-release handling, miss detection, and feedback updates.

Using the scheduled beat time rather than resetting the timer to the current time avoids accumulating timing drift. The while loop also catches up if a frame takes longer than expected.

How the timing works

Default BPM

120

Time per beat

0.5 seconds

0.0 s

Beat 0

0.5 s

Beat 1

1.0 s

Beat 2

1.5 s

Beat 3

2.0 s

Beat 4

Notes spawn at each scheduled beat, while the existing random lane selection determines which lane receives each note. Both normal and hold notes remain supported.

Test Task 3

Run the game:

```
python main.py
```

Verify the following:

- Notes spawn at approximately 0.5-second intervals at 120 BPM.
- Notes continue to use random lanes.
- Both normal and hold notes spawn.
- Hold notes still require a one-second hold.
- PERFECT, GREAT, and OK sounds still work.
- Existing scoring, misses, controls, and restart behavior remain unchanged.
- No Task 4 summary screen has been added.

After testing, commit this task separately:

```
git add game/game_engine.py
```

```
git commit -m "Add BPM-synced note spawning"
```

Status: The Task 3 code changes are provided but have not been run against your local project. Before replacing the old timer logic, confirm its exact location in your current `update()` method so that the hold-note and miss-handling logic stays intact.

Implement Task 4 from the README.

Task 4: Grade Summary Screen

After game over, display:

- PERFECT count
- GREAT count
- OK count
- MISS count
- Accuracy percentage

Accuracy should be:

$\text{successful hits} / \text{total judged notes} \times 100$

Requirements:

- Preserve Tasks 1–3.
- Track PERFECT, GREAT, OK and MISS.
- Avoid division by zero.
- Show the summary after game over.
- Allow restart using R.
- No external dependencies.
- Make the smallest necessary changes.

Worked for 13s

Reviewed rhythm-tap tasks, decisions, source files, and sound code

Task 4: Grade Summary Screen

We'll make the smallest necessary changes to `game/game_engine.py`. The summary will appear when the game ends, show all four grade counts and accuracy, and allow the player to restart with R. Tasks 1–3 remain unchanged.

1. Initialize the grade counters

Inside `reset()`, add these counters so they are initialized at the beginning of each game and reset when the player presses R.

```
# Task 4: Grade counters
self.perfect_count = 0
self.great_count = 0
self.ok_count = 0
self.grade_miss_count = 0
```

2. Add a grade-recording method

Add this method inside the `GameEngine` class:

```
def record_grade(self, grade): """Record the result of a judged note.""" if grade == "PERFECT":
self.perfect_count += 1 elif grade == "GREAT": self.great_count += 1 elif grade == "OK":
self.ok_count += 1 elif grade == "MISS": self.grade_miss_count += 1
```

3. Record every judged note

In `process_tap()`, call `record_grade()` after determining the grade for a successful normal note:

```
self.record_grade(grade)
```

In the unsuccessful tap branch, add:

```
self.record_grade("MISS")
```

Also add the corresponding calls to your existing hold-note logic:

- When a hold completes successfully: `self.record_grade("PERFECT")`
- When a hold is released too early: `self.record_grade("MISS")`
- When a note passes the hit window without being hit: `self.record_grade("MISS")`

Each note should be recorded exactly once when its outcome is determined. Keep your existing score, combo, sound-effect and game-over logic intact.

4. Display the grade summary

Add this method inside `GameEngine`:

```
def draw_grade_summary(self): overlay = pygame.Surface( (WIDTH, HEIGHT),
pygame.SRCALPHA ) overlay.fill((0, 0, 0, 210)) self.screen.blit(overlay, (0, 0)) total_judged = (
self.perfect_count + self.great_count + self.ok_count + self.grade_miss_count )
successful_hits = ( self.perfect_count + self.great_count + self.ok_count ) accuracy = (
successful_hits / total_judged * 100 if total_judged > 0 else 0.0 ) title =
self.big_font.render( "GRADE SUMMARY", True, (255, 255, 255) ) self.screen.blit(
title, (WIDTH // 2 - title.get_width() // 2, 110))
```

The accuracy formula is:

$$\left[\begin{array}{l} \text{Accuracy} = \begin{cases} \frac{\text{PERFECT} + \text{GREAT} + \text{OK}}{\text{PERFECT} + \text{GREAT} + \text{OK} + \text{MISS}} \times 100, & \text{if total judged} > 0 \\ 0, & \text{otherwise} \end{cases} \end{array} \right]$$

This avoids division by zero.

5. Show the summary after game over

At the end of your existing `draw()` method, before `pygame.display.flip()`, add:

```
if self.game_over: self.draw_grade_summary()
```

Your existing event handler should continue to process R by calling `self.reset()`. Ensure gameplay input remains disabled while `self.game_over` is true, except for restart.

6. Test and commit

Run the game:

python main.py

Verify that:

- The summary appears after game over.
- PERFECT, GREAT, OK and MISS counts update correctly.
- Accuracy uses successful hits divided by all judged notes.
- Accuracy displays 0.00% if no notes were judged.
- Pressing R resets the counters and starts a new game.
- Tasks 1–3 still work.

Commit Task 4 separately after testing:

```
git add game/game_engine.py
```

```
git commit -m "Add grade summary screen"
```

Status: The Task 4 implementation is provided, but it has not been executed against your local project. The changes above are intended to be integrated into your existing Tasks 1–3 code, not to replace the entire game engine.